

COMS BC1016

Introduction to Computational Thinking and Data Science

# Lecture 4: Data Visualizations

Sept 30, 2025

Copyright © 2026 Barnard College

September 22, 2026

# Lab Updates

- Registrar accidentally double-booked Section 2's room. New location is in Barnard Hall:
  - Section 1: Wednesday 2:40-3:55 (LL001 Milstein)
  - Section 2: Wednesday 4:10-5:25 (409 Barnard Hall)
  - Section 3: Thursday 9:40-11 (516 Milstein)
  - Section 4: Wednesday 4:10-5:25 (LL107 R&D Science Center)

# Update to Lab Grading

- Lab autograder assigns on a 100 point scale...
- We're updating grading to be out of 200 points
  - Up to 100 points for assignment correctness/completion
  - Up to 100 points for attendance
- Functionally equivalent to our previous grading scheme :)

# Homeworks

- HW 1 has been officially assigned
  - Due in 1.5 weeks on Wednesday, 9/30
- You may see some functions that we have not (and will not) cover in class
  - Assignment (and documentation) should give you enough information
  - Please come to office hours if you need help!

# Office Hours

- TA office hours are held in [Milstein 502](#), the large CS/math help room
- There may be occasional time/location updates, which will be posted on EdStem
- If you are not able to make any of the office hours or need extra office hours, feel free to reach out for extra office hours
- Contacts are on the course website

**Last Time: Arrays and Tables**

# Last Time: Arrays

- **Arrays** are sequences of values

(Note: in this class we'll mostly use NumPy arrays)

- **Examples:**

- `[1, 2, 3, 5]`

- `["Mystery", "Abby", "Jinu", "Baby", "Romance"]`

- `['apples', 'bananas', 'apples', 'apples']`

- **Elements of an array should have the same type**

- **Can make arrays using** `datascience.make_array` **or** `numpy.array`

# Last Time: Ranges

A **range** is an array of **consecutive numbers**:

- `np.arange(end)`

Create an array of increasing integers from 0 up to `end`

- `np.arange(start, end)`

Create an array of increasing integers from `start` up to `end`

- `np.arange(start, end, step)`

A range where `step` is added between consecutive values

The range always includes `start` but *excludes* `end`

# Last Time: Tables

A **table** is a way of representing data sets

- Each **row** is an **individual**
- Each **column** is an **attribute** of the individual
- Table columns are **arrays**

| Name     | Age    | Coloring             | Favorite Food |
|----------|--------|----------------------|---------------|
| Gertrude | 15 yrs | Tuxedo               | Milk          |
| Ruby     | 14 yrs | Tuxedo               | Potato chips  |
| Corina   | 6 yrs  | Dilute Tortoiseshell | Kibble        |
| Frito    | 1 yr   | Tabby                | Cheese        |

# Creating a datascience Tables

- Read from a CSV file
  - `Table.read_table(filename)`
- Create an empty table using `Table()`
  - Add elements to the Table using `.with_column`

| Name                                                                                             | Description                                                                                                                                                    | Input                                                                                         | Output                                                                 |
|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| <code>Table()</code>                                                                             | Create an empty table, usually to extend with data ( <a href="#">Ch 6</a> )                                                                                    | None                                                                                          | An empty <b>Table</b>                                                  |
| <code>Table().read_table(filename)</code>                                                        | Create a table from a data file ( <a href="#">Ch 6</a> )                                                                                                       | <b>string</b> : the name of the file                                                          | <b>Table</b> with the contents of the data file                        |
| <code>tbl.with_columns(name, values)</code><br><code>tbl.with_columns(n1, v1, n2, v2,...)</code> | A table with an additional or replaced column or columns. <b>name</b> is a string for the name of a column, <b>values</b> is an array ( <a href="#">Ch 6</a> ) | 1. <b>string</b> : the name of the new column;<br>2. <b>array</b> : the values in that column | <b>Table</b> : a copy of the original Table with the new columns added |

# More Ways to Create Tables

Create a new table from an existing table. Let `tbl` be a table and `c`, `c1`, `c2` be column names or indices

- Create a table with only the specified columns

```
tbl.select(c1, c2, ...)
```

- Copy the original table but *without* specified columns

```
tbl.drop(c1, c2, ...)
```

- Copy the original table but only with individuals in specified rows

```
tbl.take(row_indices)
```

Note that all of these produce `Tables`

# More Ways to Create Tables

Create a new table from an existing table. Let `tbl` be a table and `c`, `c1`, `c2` be column names or indices

- Copy the original table but sorted by column `c`

```
tbl.sort(c[, descending=False])
```

- Copy the original table but only with individuals where their value in `c` meets some predicate

```
tbl.where(c, predicate)
```

Note that all of these produce `Tables`

# Table Columns are Arrays

Each column in a Table is an array

- `column` takes a label or index and returns an array
- Array methods work on data in the columns
  - e.g., `sum`, `min`, `max`, `average`

# Demo: NBA Salaries

# Attributes

# Attribute Types

- Table columns are attributes
- All values in a column should be the same type *and* comparable to each other
  - **Numerical:** Values are on a numerical scale (e.g., years)
    - Numerical values are ordered
    - Differences are meaningful
  - **Categorical:** Each value is from a fixed inventory (e.g., material)
    - May or may not have an ordering
    - Categories are either the same or different

# Numerical Attributes

Values that are numbers are not necessarily numerical

- Sometimes people use numbers instead of strings to represent categories
- Example: In US census data, `SEX` code is (0, 1, 2)
  - Arithmetic on these “numbers” is meaningless
  - The variable `SEX` is still categorical even though numbers were used for the categories

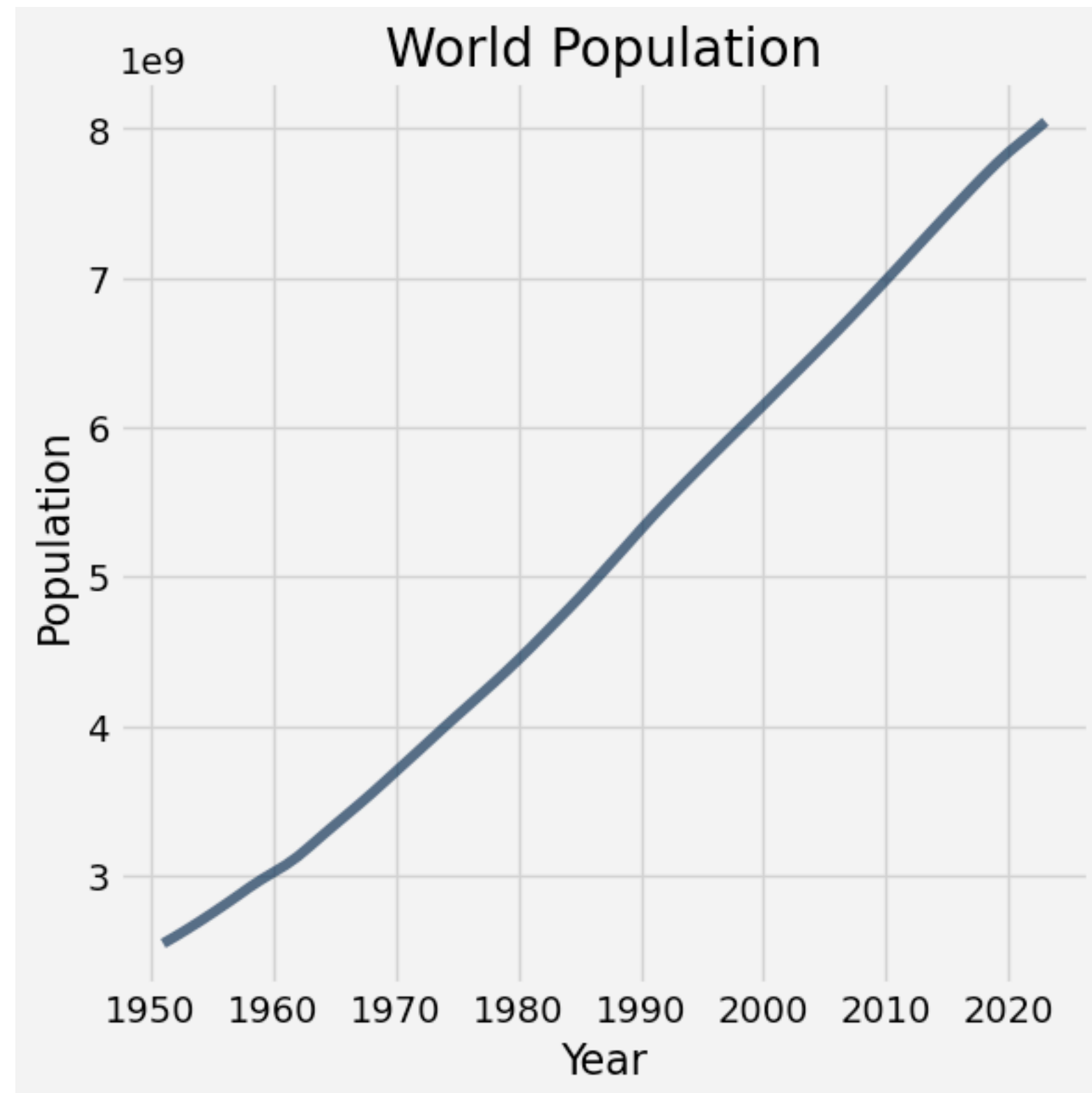
DEMO

# Numerical Data

# Plotting Two Numerical Variables

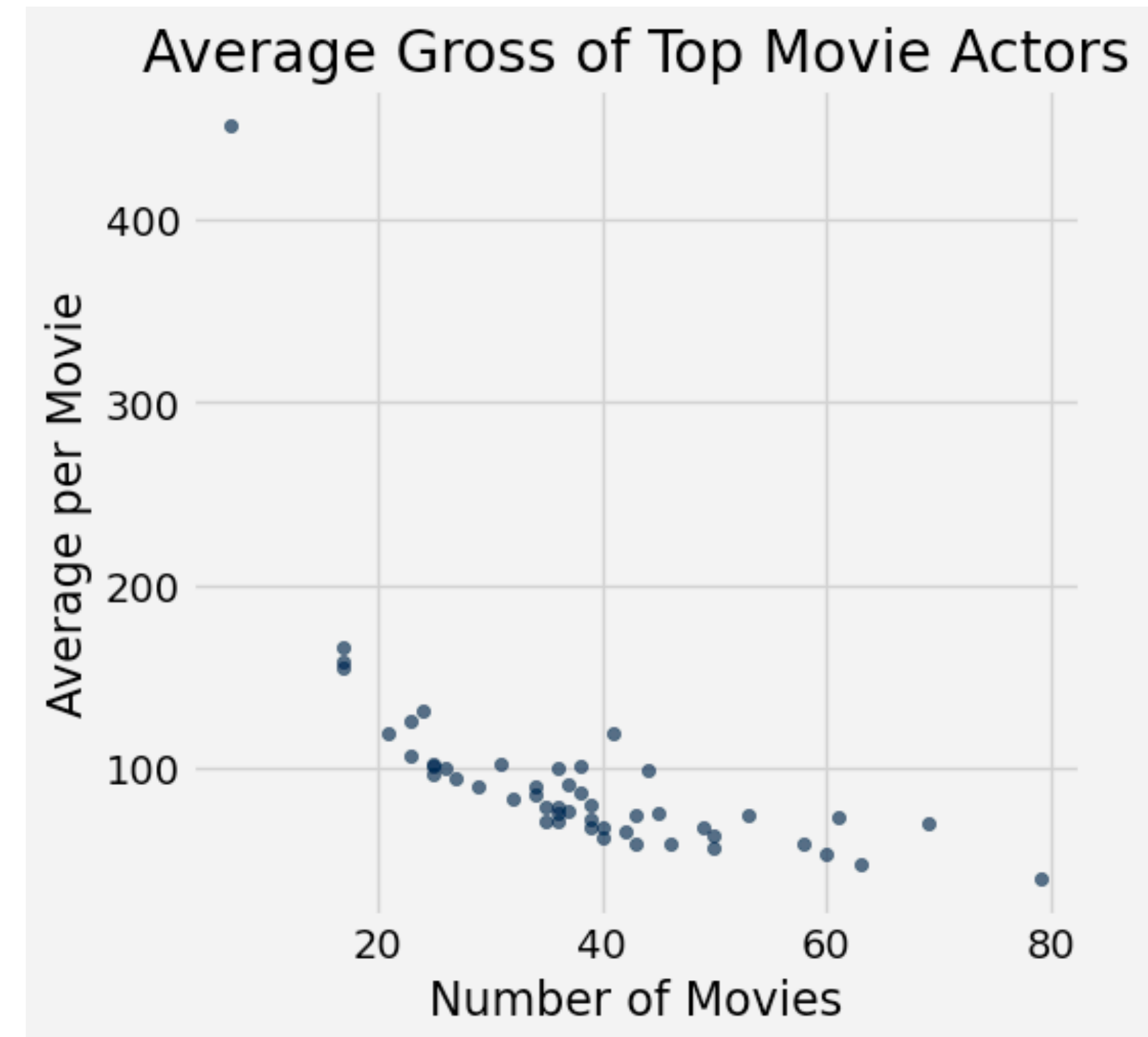
Line Plot

`plot`



Scatter Plot

`scatter`



# Line Plots vs Scatter Plots

```
tbl.plot(x_label, y_label)
```

```
tbl.scatter(x_label, y_label)
```

- Use **line plots** for **sequential** data if:
  - x-axis has an order (e.g., time, years, distance)
  - sequential differences in y value are meaningful
  - there's only one y-value for each x-value
- Use **scatter plots** for **non-sequential** quantitative data if:
  - looking for associations

# Demo: Line and Scatter Plots

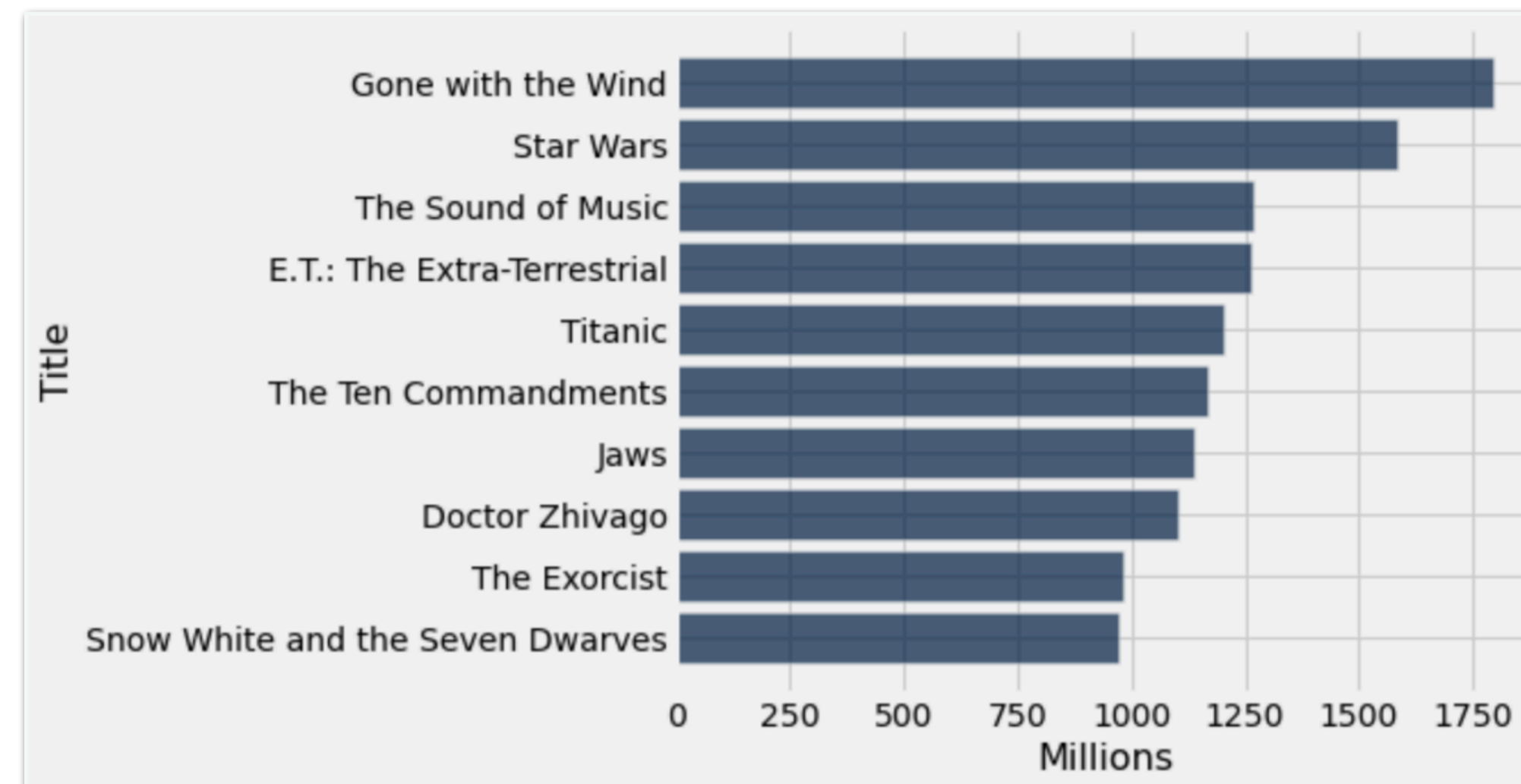
# Categorical and Numerical Attributes

# Bar Charts for Categorical Attributes

```
tbl.barh(category_label, freq_label)
```

Bar charts are good for visualizing:

- relationships between categorical variable(s) and a numerical variable



# Demo: Bar Charts

# Next Class

- Today
  - Charts and Visualizations
- Wednesday
  - Visualizing categorical distributions (Bar chart)
  - Visualizing numerical distributions (Histograms)